

Tarea 1: Análisis de Algoritmos y Notaciones Asintóticas

Análisis y Diseño de Algoritmos - CC3041

José Antonio Mérida Castejón

Jonathan Alejandro Díaz Tahuite

Luis Francisco Padilla Juarez

8 de febrero de 2026

1. Describa un algoritmo de tiempo de ejecución $O(n \log_2 n)$ tal que, dados un conjunto S de n números enteros y un entero arbitrario x , determine si existen o no dos números en S cuya suma sea exactamente x . Puede suponer que el arreglo está ordenado.

El approach más eficiente sería utilizar dos punteros, dónde

```
while i < j:
    if S[i] + S[j] == x: return true
    if S[i] + S[j] < x: i++
    if S[i] + S[j] > x: j--
```

Esto nos daría una complejidad de $O(n)$, si fuésemos a tomar el ordenamiento en cuenta sería $O(n \log_2 n)$ ya que el ordenamiento predomina.

Por otro lado, si quisiéramos una solución de $O(n \log_2 n)$ para una entrada ya ordenada, podríamos iterar sobre cada elemento $S[i]$ y buscar su complemento ($x - S[i]$) utilizando una búsqueda binaria en el resto del arreglo.

```
for i = 0 to n - 1:
    target = x - S[i]
    if BinarySearch(S, target, i + 1, n - 1):
        return true
return false
```

La búsqueda binaria detallada a continuación tiene complejidad de $O(\log_2 n)$

```

BinarySearch(S, k, low, high):
    while low ≤ high:
        mid = floor((low + high) / 2)
        if S[mid] == k: return true
        else if S[mid] < k: low = mid + 1
        else: high = mid - 1
    return false

```

Al realizarlo para cada uno de los elementos, la complejidad es de $O(n \log_2 n)$.

2. La Regla de Horner dice que se puede evaluar un polinomio $P(x) = \sum_{k=0}^n a_k x^k$ de la siguiente manera:

$$a_0 + x(a_1 + x(a_2 + \cdots + x(a_{n-1} + xa_n)\dots))$$

El siguiente pseudocódigo implementa esta regla:

```

y = 0
for i = n downto 0:
    y = a[i] + x * y

```

Calcule una cota ajustada para el tiempo de ejecución de este algoritmo.

Tenemos un bucle principal que corre $n + 1$ veces y su contenido es una suma y una multiplicación $O(1)$ que ocurren cada ciclo. Por lo tanto, la cota ajustada es $\Theta(n)$.

3. Escriba código naive para la evaluación de un polinomio (suponga que no hay una instrucción primitiva para calcular x^y). Compare las tasas de crecimiento de este código y el que implementa la Regla de Horner.

```

y = 0
for k = 0 to n:
    potencia = 1
    for j = 1 to k:
        potencia = potencia * x
    y = y + a[k] * potencia

```

Para este algoritmo, podemos darnos cuenta que cada término de potencia p nos lleva a realizar p cálculos para evaluarlo individualmente. Entonces, tenemos n términos a evaluar con un total de operaciones:

$$\sum_{i=1}^n i = n(n+1)/2$$

Y por lo tanto una complejidad de $O(n^2)$. La implementación utilizando la Regla de Horner tiene una tasa de crecimiento lineal, por lo que es mucho más eficiente que nuestra implementación naive.

4. Para dos funciones $f(n)$ y $g(n)$, demuestre que $\max(f(n), g(n)) = \Theta(f(n) + g(n))$.

Demostración. Sean $f(n)$ y $g(n)$ funciones asintóticamente positivas. Por lo tanto, existen n_1, n_2 tales que $f(n) \geq 0, \forall n \geq n_1$ y $g(n) \geq 0, \forall n \geq n_2$.

1. Cota Superior (O):

Sea $n_0 = \max(n_1, n_2)$, entonces:

$$\begin{aligned} 0 &\leq f(n) \leq f(n) \text{ y } 0 \leq g(n) \leq g(n) \\ \Rightarrow 0 &\leq f(n) \leq f(n) + g(n) \text{ y } 0 \leq g(n) \leq f(n) + g(n) \\ \Rightarrow \max(f(n), g(n)) &\leq f(n) + g(n), \forall n > n_0 \end{aligned}$$

Al encontrar una constante positiva $c = 1$ que acota superiormente la función:

$$\therefore \max(f(n), g(n)) \in O(f(n) + g(n))$$

2. Cota Inferior (Ω):

Por definición de máximo, sabemos que $f(n) \leq \max(f(n), g(n))$ y $g(n) \leq \max(f(n), g(n))$.

Sumando ambas desigualdades:

$$\begin{aligned} f(n) + g(n) &\leq \max(f(n), g(n)) + \max(f(n), g(n)) \\ \Rightarrow f(n) + g(n) &\leq 2 \max(f(n), g(n)) \\ \Rightarrow \frac{1}{2}(f(n) + g(n)) &\leq \max(f(n), g(n)) \end{aligned}$$

Al encontrar una constante positiva $c = \frac{1}{2}$ que acota inferiormente la función:

$$\therefore \max(f(n), g(n)) \in \Omega(f(n) + g(n))$$

Al haber demostrado la existencia de una cota superior e inferior, concluimos que:

$$\therefore \max(f(n), g(n)) \in \Theta(f(n) + g(n)) \blacksquare$$

5 Argumente por que, para constantes reales cualquiera a y $b > 0$, $(n + a)^b = \Theta(n^b)$

Enunciado: a y $b > 0$, $(n + a)^b = \Theta(n^b)$

Forma Expandida (Hint):

Utilizando una generalización del Teorema del Binomio se puede expresar la función como:

$$(n + a)^b = \left[n \left(1 + \frac{a}{n} \right) \right]^b = n^b \left(1 + \frac{a}{n} \right)^b$$

Si expandimos el término $\left(1 + \frac{a}{n} \right)^b$ usando series de Taylor, se obtiene:

$$(n + a)^b = n^b \left(1 + b \left(\frac{a}{n} \right) + \frac{b(b-1)}{2!} \left(\frac{a}{n} \right)^2 + \dots \right)$$

$$(n + a)^b = n^b + abn^{b-1} + \dots$$

En el análisis, los términos de orden inferior ($n^{b-1}, n^{b-2}, \dots$) se vuelven insignificantes frente al término dominante n^b cuando $n \rightarrow \infty$.

Prueba mediante Límite:

Se calcula el límite del cociente entre la función y su propuesta asintótica:

$$L = \lim_{n \rightarrow \infty} \frac{(n + a)^b}{n^b} = \lim_{n \rightarrow \infty} \left(\frac{n + a}{n} \right)^b = \lim_{n \rightarrow \infty} \left(1 + \frac{a}{n} \right)^b$$

Sabemos que $\lim_{n \rightarrow \infty} \frac{a}{n} = 0$ para cualquier constante real a .

$$L = (1 + 0)^b = 1$$

Dado que el límite es una constante positiva $0 < L < \infty$, por definición de Θ :

$$(n + a)^b = \Theta(n^b)$$

6 ¿Es $2^{n+1} = O(2^n)$? ¿Es $2^{2n} = O(2^n)$?

Enunciado: ¿Es $2^{n+1} = O(2^n)$? ¿Es $2^{2n} = O(2^n)$?

Parte A: $2^{n+1} = O(2^n)$

Demostración:

Se busca $c > 0, n_0$ tales que $2^{n+1} \leq c \cdot 2^n$.

$$2^{n+1} = 2^1 \cdot 2^n = 2 \cdot 2^n$$

Si se selecciona $c \geq 2$ y $n_0 = 1$:

$$2 \cdot 2^n \leq c \cdot 2^n$$

La desigualdad se mantiene.

Respuesta:

Si, $2^{n+1} = O(2^n)$.

Parte B: $2^{2n} = O(2^n)$

Demostración:

Se simplifica la expresion izquierda:

$$2^{2n} = (2^2)^n = 4^n$$

Por lo que:

$$4^n = O(2^n)$$

Entonces $\exists c$ tal que $4^n \leq c \cdot 2^n$.

Dividiendo por 2^n :

$$\frac{4^n}{2^n} \leq c \implies 2^n \leq c$$

Esto implica que la función 2^n está acotada por una constante c , lo cual es falso ya que $\lim_{n \rightarrow \infty} 2^n = \infty$.

Respuesta:

No. $2^{2n} \neq O(2^n)$

7. Demuestre las siguientes propiedades:

Inciso a

Proposición: $f(n) = \Theta(g(n)) \iff f(n) = O(g(n)) \wedge f(n) = \Omega(g(n))$.

Demostración:

($\Rightarrow$) Si es Θ , entonces es O y Ω :

Por definición de $\Theta(g(n))$, existen c_1, c_2, n_0 tales que para todo $n \geq n_0$:

$$0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n)$$

La parte derecha $f(n) \leq c_2 g(n)$ satisface la definición de $O(g(n))$.

La parte izquierda $c_1 g(n) \leq f(n)$ satisface la definición de $\Omega(g(n))$.

($\Leftarrow$) Si es O y Ω , entonces es Θ :

$$f(n) = O(g(n)) \implies f(n) \leq c_2 g(n) \text{ para } n \geq n_a.$$

$$f(n) = \Omega(g(n)) \implies f(n) \geq c_1 g(n) \text{ para } n \geq n_b.$$

Sea $n_0 = \max(n_a, n_b)$. Para $n \geq n_0$ ambas desigualdades son ciertas simultáneamente, recuperando la definición de Θ .

Inciso b

Proposición: $o(g(n)) \cap \omega(g(n)) = \emptyset$.

Demostración por Contradicción:

Se supone que existe una función $f(n)$ tal que $f(n) \in o(g(n))$ y $f(n) \in \omega(g(n))$.

Por definición de $o(g(n))$ (MIT Lecture Notes):

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0$$

Por definición de $\omega(g(n))$:

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \infty$$

Una función bien definida no puede tender a 0 y a ∞ al mismo tiempo cuando $n \rightarrow \infty$.

$$\therefore o(g(n)) \cap \omega(g(n)) = \emptyset$$

Inciso c

Proposición: $f(n) = O(g(n)) \implies \log_2 f(n) = O(\log_2 g(n))$.

Demostración:

Existe c, n_0 tal que $f(n) \leq c \cdot g(n)$ para $n \geq n_0$.

Se aplica $\log_2$

$$\log_2 f(n) \leq \log_2 (c \cdot g(n))$$

$$\log_2 f(n) \leq \log_2 c + \log_2 g(n)$$

Para demostrar que es $O(\log_2 g(n))$, se necesita acotar $\log_2 c$ en terminos de $\log_2 g(n)$.

Dado que $\log_2 g(n) \geq 1$, cualquier constante $K = \log_2 c$ puede ser acotada por $|K| \cdot \log_2 g(n)$.

Sustituyendo:

$$\log_2 f(n) \leq |K| \cdot \log_2 g(n) + 1 \cdot \log_2 g(n)$$

$$\log_2 f(n) \leq (|K| + 1) \log_2 g(n)$$

Sea la nueva constante $C' = |K| + 1$.

$$\log_2 f(n) \leq C' \cdot \log_2 g(n)$$

$$\therefore \log_2 f(n) = O(\log_2 g(n))$$